

Homework 03 — Ridge Regression, Shrinkage and Penalty Selection

Eric Chen (ec-03) · All computations in R (base); seed 43231 (Q2). Scaling throughout: each predictor centred to mean zero and divided by $\sqrt{\text{mean squared deviation}}$, i.e. variance with divisor n ; the intercept is fitted as $\bar{y}$ and never penalised.

Question 1 · Building the explain-to-me skill

I wrote `explain-to-me/SKILL.md` as a short, single-purpose skill and installed it at project scope in `.claude/skills/explain-to-me/SKILL.md`, which is the agent's normal installation path for a repository-local skill. Its complete contents are reproduced in the appendix on the last page, as required.

Three design decisions did the real work. First, the **description** field states not only when to use the skill but when *not* to — it excludes requests for a solution, for code, or for a check of finished work — because a description that only says “helps with homework” invites the agent to activate on every question I ask and take over the assignment. Second, the instructions fix an **output shape** rather than a topic: a two-or-three-sentence statement of the goal, then one line per lettered part naming the deliverable (derivation, number, table, plot, or interpretation). That converts a vague “explain this” into something checkable, and it is the part I found most useful in practice. Third, the reading pointers are mapped by **subject** rather than by question number — shrinkage, collinearity and bias-variance go to *Ridge Regression: Stability Through Shrinkage*, and optimisation, cross-validation, GCV and degrees of freedom go to *From a Penalized Objective to a Fitted Ridge Model* — with an explicit instruction to say so rather than invent a section if the page cannot be opened.

Test on Question 2. Reading Question 2 through the skill produced exactly the prescribed shape. The goal came back in three sentences — the question builds one fixed, nearly collinear design and then resamples only the noise, so as to separate what happens to the coefficients from what happens to the fitted values — followed by one line per part: (a) three numbers and an argument connecting them, (b) a table of means and standard deviations over 200 repetitions plus two averaged MSEs, (c) an interpretation needing no new computation. It defined the scaling convention as the divisor- n version rather than the usual sample standard deviation, warned that $\mathbf{X}$ must not be regenerated inside the loop and that the pair (ϵ, ϵ^*) is drawn once per repetition and reused across all three values of λ , noted that the test MSE uses the *training* intercept $\bar{y}$, and pointed at the common-scale section of the shrinkage page. It closed by suggesting that I inspect the singular values of $\mathbf{X}$ before writing any loop — which is in fact where part (a) is decided — and asked which part was still unclear. It computed nothing. Two practical notes: the skill file has to be in place before the agent loads its skill list, so a skill added part-way through a session is not visible until the next one; and the **Restrictions** block is what keeps the explanation from sliding into a solution, since every other rule in the file is phrased as something to do rather than something to withhold. The skill did not interfere with unrelated requests, so I left it installed.

Question 2 · Ridge regression with nearly collinear predictors

```
set.seed(43231)
L <- rnorm(n); U1 <- rnorm(n); U2 <- rnorm(n); X3 <- rnorm(n); X4 <- rnorm(n)
Xraw <- cbind(L + 0.04*U1, L + 0.04*U2, X3, X4) # then centre, scale to mean sq 1
ridge <- function(X, y, lam) solve(crossprod(X) + nrow(X)*lam*diag(ncol(X)),
                                crossprod(X, y - mean(y))) # b0 = mean(y)
```

(a) Collinearity diagnostics and the two contrasts

quantity	value	quantity	value
$\text{cor}(\mathbf{x}_1, \mathbf{x}_2)$	0.99771	$\text{Var}(\hat{\beta}_1 + \hat{\beta}_2) / \sigma^2$	0.01002
$d_{\max} = d_1$	14.1415	$\text{Var}(\hat{\beta}_1 - \hat{\beta}_2) / \sigma^2$	8.8506
$d_{\min} = d_4$	0.47536	variance ratio	883.1
$d_{\max}/d_{\min}$	29.749	full spectrum	14.14, 10.35, 9.63, 0.475

For ordinary least squares on a fixed design, $\text{Var}(\mathbf{c}^T \boldsymbol{\beta}) = \sigma^2 \mathbf{c}^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{c} = \sigma^2 \sum_j (\mathbf{c}^T \mathbf{v}_j)^2 / d_j^2$, where $\mathbf{v}_j$ are the right singular vectors. The realised design puts the two contrasts at opposite ends of that sum. The **weakest** singular direction is $\mathbf{v}_4 = (0.7071, -0.7071, 0.000, 0.000)^T$, which is the difference direction $(1, -1, 0, 0)/\sqrt{2}$ to three decimals; the **strongest**, $\mathbf{v}_1 = (-0.7064, -0.7063, -0.046, 0.004)^T$, is essentially the sum direction. So the difference contrast is divided by $d_4^2 = 0.226$ while the sum contrast is divided by $d_1^2 = 200.0$, giving $8.85\sigma^2$ against $0.0100\sigma^2$ — a variance ratio of **883**, so $\beta_1 - \beta_2$ has about 30 times the standard error of $\beta_1 + \beta_2$.

The mechanism is the construction itself. Both $\mathbf{x}_1$ and $\mathbf{x}_2$ equal the shared factor $\mathbf{L}$ plus a 0.04-scale idiosyncratic term, so they correlate at 0.998. The data therefore pin down *how much of the shared factor enters the mean*, which is what $\beta_1 + \beta_2$ measures, but carry almost no information about *how that total is split* between two nearly identical columns. Moving along $(1, -1, 0, 0)$ barely changes $\mathbf{X}\boldsymbol{\beta}$, so the residual sum of squares is nearly flat in that direction and the fit is free to wander along it under noise.

(b) Two hundred repetitions on the fixed design

$\mathbf{X}$ is generated once and held fixed. Inside each of the 200 repetitions a fresh pair $(\mathbf{y}, \mathbf{y}^*)$ is drawn and that same realised pair is reused for all three penalties, so the three columns below are directly comparable run by run.

λ	β_1		β_2		$\beta_1 + \beta_2$		$\beta_1 - \beta_2$		avg MSE	avg MSE
	mean	sd	mean	sd	mean	sd	mean	sd	train	test
0 (OLS)	1.4956	1.5439	1.5098	1.5486	3.0054	0.0951	-0.0142	3.0911	0.9648	1.0625
0.02	1.4870	0.1615	1.4892	0.1659	2.9762	0.0942	-0.0022	0.3136	0.9747	1.0493
0.20	1.3679	0.0457	1.3687	0.0473	2.7366	0.0865	-0.0008	0.0344	1.0768	1.1335
truth	1.5	—	1.5	—	3.0	—	0.0	—	—	—

Monte Carlo check: the theoretical OLS values from part (a) are $\text{sd}(\beta_1 + \beta_2) = 0.1001$ and $\text{sd}(\beta_1 - \beta_2) = 2.975$, both within the sampling error of 200 replications.

(c) Interpretation through bias and variance

The coefficients. At $\lambda = 0$ the individual estimates are unbiased — 1.496 and 1.510 against a truth of 1.5 — but useless: a standard deviation of 1.54 means a typical run reports values anywhere from roughly -1.5 to 4.5, and in some runs the sign of a coefficient with true value 1.5 is negative. Everything that is wrong lives in the contrast $\beta_1 - \beta_2$, whose standard deviation is 3.09 around a true value of 0, while the sum is already accurate to 0.095 without any penalty. A small penalty fixes exactly this: at $\lambda = 0.02$ the standard deviation of each coefficient drops by a factor of 9.6, to 0.162, and the difference contracts from 3.09 to 0.314, at a cost in bias of only 0.9 % (mean 1.487 against 1.5). At $\lambda = 0.2$ variance falls further, to 0.046, but the bias is now real: the coefficients sit at 1.368 and the sum at 2.737, an 8.8 % shortfall, because the penalty has started pulling on the well-determined sum direction too.

The test MSE. It moves far less than the coefficients do: $1.0625 \rightarrow 1.0493 \rightarrow 1.1335$. The irreducible noise contributes $\sigma^2 = 1$ to every entry, so the interesting part is the excess over 1: 0.0625, 0.0493 and 0.1335. On that scale $\lambda = 0.02$ removes 21 % of the excess prediction error while $\lambda = 0.2$ more than doubles it — the usual U shape, with a shallow optimum near $\lambda = 0.02$. Training MSE, by contrast, rises monotonically (0.965, 0.975, 1.077), as it must: least squares minimises it by definition, so any penalty can only make it worse.

Why the coefficients change so much more than the fit. The singular value decomposition separates the two. Writing $\mathbf{X} = \sum_j d_j \mathbf{u}_j \mathbf{v}_j^T$, ridge multiplies the fitted component along $\mathbf{u}_j$ by $\rho_j(\lambda)$ but produces the coefficient by dividing that same component by d_j :

$$\mathbf{X}\boldsymbol{\beta}_\lambda = \sum_j \rho_j(\lambda) \mathbf{u}_j (\mathbf{u}_j^T \tilde{\mathbf{y}}), \quad \beta_\lambda = \sum_j \rho_j(\lambda) \frac{\mathbf{u}_j^T \tilde{\mathbf{y}}}{d_j} \mathbf{v}_j.$$

j	d_j	$\rho_j(0.02)$	$\rho_j(0.2)$	share of $\ \mu\ ^2$	$(v_j)_1$	$(v_j)_2$
1	14.141	0.9901	0.9091	92.05 %	-0.7064	-0.7063
2	10.348	0.9817	0.8426	4.13 %	0.0221	0.0275
3	9.629	0.9789	0.8226	3.82 %	0.0245	0.0180
4	0.4754	0.1015	0.0112	8×10^{-11}	0.7071	-0.7071

At $\lambda = 0.02$ the three well-determined directions keep at least 97.9 % of their fitted component, so $\mathbf{X}\beta$ hardly moves; only the fourth direction is cut, by 90 %. That direction carries 8×10^{-11} of $\|\mu\|^2$ — the true $\beta = (1.5, 1.5, 1, 0)$ is orthogonal to it — so discarding it costs essentially no bias in the fitted values. In the coefficient expansion, however, that same term is divided by $d_4 = 0.475$, which amplifies the noise projection $\mathbf{u}_4^T \varepsilon$ by a factor of 2.1 and then loads it onto β_1 and β_2 with weights ± 0.707 — equal in size and opposite in sign. Suppressing one near-null direction therefore removes almost all of the coefficient variance while subtracting almost nothing from the fit. In degrees-of-freedom terms the penalty is remarkably cheap: df_{eff} falls only from 5 at $\lambda = 0$ to 4.052 at $\lambda = 0.02$, and nearly the whole of that one degree of freedom is taken from direction 4. At $\lambda = 0.2$ ($\text{df}_{\text{eff}} = 3.585$) the shrinkage has spread to $\rho_1 = 0.909$, which bites into the 92 % of the signal held in direction 1 — that is precisely the bias visible in the sum, and why the test MSE turns back up.

Question 3 · What does ridge regression shrink?

(a) The ridge normal equation and its unique solution

With $L_\lambda(\beta) = (1/2n)\|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + (\lambda/2)\|\beta\|_2^2$, both terms are differentiable, so

$$\nabla L_\lambda(\beta) = -\frac{1}{n} \mathbf{X}^T(\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda\beta = \frac{1}{n}(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I}_p)\beta - \frac{1}{n}\mathbf{X}^T\tilde{\mathbf{y}}.$$

Setting the gradient to zero and multiplying through by n gives $(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I}_p)\beta_\lambda = \mathbf{X}^T\tilde{\mathbf{y}}$, as claimed. The Hessian is $\nabla^2 L_\lambda = (1/n)\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}_p$, constant in β , so the stationary point is the global minimiser.

Uniqueness. $\mathbf{X}^T\mathbf{X}$ is symmetric positive semi-definite, so its eigenvalues satisfy $v_1 \geq \dots \geq v_p \geq 0$. Adding $n\lambda\mathbf{I}_p$ leaves the eigenvectors unchanged and shifts every eigenvalue up by the same amount, giving $v_j + n\lambda \geq n\lambda > 0$ whenever $\lambda > 0$. Every eigenvalue is then strictly positive, so $\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I}_p$ is positive definite, hence invertible, and $\beta_\lambda = (\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I}_p)^{-1}\mathbf{X}^T\tilde{\mathbf{y}}$ is the one and only solution. Nothing in this argument refers to the rank of $\mathbf{X}$ or to the relative size of n and p : if $\mathbf{X}$ is rank deficient or $p > n$, then $\mathbf{X}^T\mathbf{X}$ is singular and least squares has infinitely many solutions, but the shift by $n\lambda$ still lifts the zero eigenvalues off zero. Equivalently, the penalty makes L_λ strictly convex, and a strictly convex function has at most one minimiser.

(b) Shrinkage in the singular directions

Let $\mathbf{X} = \mathbf{U}_r\mathbf{D}_r\mathbf{V}_r^T$ with $\mathbf{U}_r^T\mathbf{U}_r = \mathbf{V}_r^T\mathbf{V}_r = \mathbf{I}_r$, and complete $\mathbf{V}_r$ with an orthonormal basis $\mathbf{V}_\perp$ of the null space of $\mathbf{X}$. Then $\mathbf{X}^T\mathbf{X} = \mathbf{V}_r\mathbf{D}_r^2\mathbf{V}_r^T$, and since $\mathbf{V}_r\mathbf{V}_r^T + \mathbf{V}_\perp\mathbf{V}_\perp^T = \mathbf{I}_p$,

$$(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{I}_p)^{-1} = \mathbf{V}_r(\mathbf{D}_r^2 + n\lambda\mathbf{I}_r)^{-1}\mathbf{V}_r^T + (n\lambda)^{-1}\mathbf{V}_\perp\mathbf{V}_\perp^T.$$

Because $\mathbf{X}^T\tilde{\mathbf{y}} = \mathbf{V}_r\mathbf{D}_r\mathbf{U}_r^T\tilde{\mathbf{y}}$ lies in the column space of $\mathbf{V}_r$, the second term annihilates it and

$$\beta_\lambda = \mathbf{V}_r(\mathbf{D}_r^2 + n\lambda\mathbf{I}_r)^{-1}\mathbf{D}_r\mathbf{U}_r^T\tilde{\mathbf{y}}.$$

Multiplying by $\mathbf{X}$ and using $\mathbf{V}_r^T\mathbf{V}_r = \mathbf{I}_r$ gives the fitted response

$$\mathbf{X}\hat{\beta}_\lambda = \mathbf{U}_r \mathbf{D}_r^{-2} (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{U}_r^T \tilde{\mathbf{y}} = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} \mathbf{u}_j (\mathbf{u}_j^T \tilde{\mathbf{y}}).$$

At $\lambda = 0$ this is the orthogonal projection $\sum_j \mathbf{u}_j (\mathbf{u}_j^T \tilde{\mathbf{y}})$, so ridge retains exactly the fraction $\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda)$ of the least-squares component in the j th left singular direction. Since ρ_j increases in d_j and decreases in λ , directions the design explores strongly are left nearly untouched and weak directions are damped. The trace of the ridge smoother, $\sum_j \rho_j(\lambda)$, is the effective degrees of freedom of the slopes, to which the unpenalised intercept adds one.

Numerical case. With $n = 100$ and $\lambda = 0.01$ the shift is $n\lambda = 1$:

j	d_j	d_j^2	$d_j^2 + n\lambda$	$\rho_j(0.01)$	shrunk away
1	10	100	101	$100/101 = 0.990099$	1.0 %
2	2	4	5	$4/5 = 0.800000$	20.0 %
3	0.2	0.04	1.04	$0.04/1.04 = 0.038462$	96.2 %

$$\text{df}_{\text{eff}}(0.01) = 1 + (0.990099 + 0.800000 + 0.038462) = 1 + 1.828561 = \mathbf{2.828561}.$$

So a model with three predictors plus an intercept, nominally 4 parameters, behaves like about 2.83.

(c) The most shrunken direction

The strongest shrinkage falls on **direction 3**, the one with the smallest singular value $d_3 = 0.2$: it keeps only $\rho_3 = 0.0385$ of its least-squares component, discarding 96.2 %, whereas directions 1 and 2 lose 1 % and 20 %. This is structural rather than accidental — ρ_j compares d_j^2 with the fixed threshold $n\lambda$, so any direction with $d_j^2 \ll n\lambda$ is nearly eliminated.

Why this reduces prediction variance. Write $\tilde{\mathbf{y}} = \mu + \varepsilon$ with $\text{Var}(\varepsilon) = \sigma^2 \mathbf{I}_n$. Since the $\mathbf{u}_j$ are orthonormal, $\mathbf{u}_j^T \varepsilon$ has variance σ^2 in every direction — noise enters each equally. The coefficient along $\mathbf{v}_j$ is $(\mathbf{u}_j^T \tilde{\mathbf{y}}) / d_j$, with variance σ^2 / d_j^2 : that is $0.01\sigma^2$ for $j = 1$ but $25\sigma^2$ for $j = 3$, a factor of 2500. The weakest direction is where the estimate is almost pure noise, because the design supplies almost no leverage there. Ridge multiplies that direction's contribution by ρ_3 , so its variance contribution is multiplied by $\rho_3^2 = 0.00148$ — a 99.85 % reduction — and the variance of the fitted values, $\sigma^2 \sum_j \rho_j^2$, falls correspondingly. The penalty spends the least effort where the data speak clearly and the most where they do not.

When the same shrinkage causes serious bias. The bias in direction j is exactly $(1 - \rho_j)(\mathbf{u}_j^T \mu)$, so the squared bias of the fit is $\sum_j (1 - \rho_j)^2 (\mathbf{u}_j^T \mu)^2$. Whether the trade is worthwhile depends entirely on how much of the true mean lies along $\mathbf{u}_3$, which ridge cannot know: the shrinkage factor is built from d_j alone, a property of the design, and is applied blindly to whatever signal happens to sit there. If the truth is genuinely concentrated in the low-variance direction — say μ is nearly parallel to $\mathbf{u}_3$, which happens when the response depends mainly on the contrast between two nearly collinear predictors rather than on their common level — then 96.2 % of the signal is thrown away, and the squared bias $(0.962)^2 (\mathbf{u}_3^T \mu)^2$ can swamp the variance saved. That is the opposite of the Question 2 design, where the near-null direction carried 8×10^{-11} of the signal and shrinking it was nearly free. The same risk arises whenever λ is pushed high enough that $n\lambda$ approaches d_1^2 , since the leading, signal-bearing direction is then damped as well.

Question 4 · Ridge regression and optimisation

(a) Condition numbers

$\mathbf{A}_0$ has eigenvalues 9 and 0.01, and $\mathbf{A}_\lambda = \mathbf{A}_0 + \lambda \mathbf{I}$ shares its eigenvectors with every eigenvalue raised by $\lambda = 0.09$, giving 9.09 and 0.10. Hence

$$\kappa(\mathbf{A}_0) = 9 / 0.01 = \mathbf{900}, \quad \kappa(\mathbf{A}_\lambda) = 9.09 / 0.10 = \mathbf{90.9},$$

a tenfold improvement. Geometrically, the level sets of a quadratic with curvature matrix $\mathbf{A}$ are ellipses whose semi-axis in an eigendirection is proportional to $1/\sqrt{a}$, so the ratio of longest to shortest axis is $\sqrt{\kappa}$. The unpenalised objective has an axis ratio of $\sqrt{900} = 30$ — a long, narrow valley, in which the negative gradient points almost perpendicular to the direction of the minimiser and gradient descent zig-zags across the valley floor. The ridge objective has a ratio of $\sqrt{90.9} = 9.53$, a markedly rounder bowl in which the gradient points much closer to the minimiser. Note that $\lambda = 0.09$ is large relative to the small eigenvalue (it multiplies 0.01 by ten) but negligible relative to the large one ($9 \rightarrow 9.09$): the penalty acts almost entirely on the flat direction, which is exactly what makes the level sets rounder.

(b) Contraction of the optimisation error

Differentiating the ridge objective as in Question 3(a), $\nabla L_\lambda(\beta) = \mathbf{A}_\lambda \beta - (1/n)\mathbf{X}^T \tilde{\mathbf{y}}$. The minimiser satisfies $\mathbf{A}_\lambda \beta_\lambda = (1/n)\mathbf{X}^T \tilde{\mathbf{y}}$, so subtracting the two lines gives $\nabla L_\lambda(\beta^{(k)}) = \mathbf{A}_\lambda(\beta^{(k)} - \beta_\lambda) = \mathbf{A}_\lambda \mathbf{e}^{(k)}$. Subtracting β_λ from both sides of the update,

$$\mathbf{e}^{(k+1)} = \mathbf{e}^{(k)} - \eta \mathbf{A}_\lambda \mathbf{e}^{(k)} = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

$\mathbf{A}_0$ is symmetric, so it has an orthonormal eigenbasis $\mathbf{w}_1, \dots, \mathbf{w}_p$ with $\mathbf{A}_0 \mathbf{w}_j = a_j \mathbf{w}_j$, and $\mathbf{A}_\lambda \mathbf{w}_j = (a_j + \lambda) \mathbf{w}_j$. Expanding $\mathbf{e}^{(k)} = \sum_j c_j^{(k)} \mathbf{w}_j$ and applying $\mathbf{I} - \eta \mathbf{A}_\lambda$ term by term, the eigenbasis decouples the iteration into p independent scalar recursions:

$$c_j^{(k+1)} = \{1 - \eta(a_j + \lambda)\} c_j^{(k)}, \quad \text{so} \quad c_j^{(k)} = \{1 - \eta(a_j + \lambda)\}^k c_j^{(0)},$$

which is the stated per-step factor $1 - \eta(a + \lambda)$.

Admissible step sizes. Every direction contracts precisely when $|1 - \eta(a + \lambda)| < 1$ for both eigenvalues, i.e. $0 < \eta(a + \lambda) < 2$. The binding constraint is the largest eigenvalue $M = 9.09$, so

$$0 < \eta < 2/M = 2/9.09 = \mathbf{0.220022}.$$

step size	direction	$a + \lambda$	factor $1 - \eta(a + \lambda)$	behaviour
$\eta = 1/M = 0.110011$	stiff ($a = 9$)	9.09	0	error killed in one step
	flat ($a = 0.01$)	0.10	0.988999	1.1 % per step; 209 steps per decade
$\eta = 2.1/M = 0.231023$	stiff ($a = 9$)	9.09	-1.1	grows 10 % per step, alternating sign
	flat ($a = 0.01$)	0.10	0.976898	still contracting

At $\eta = 1/M$ the factor in the stiff direction is exactly $1 - 9.09/9.09 = 0$, so **the stiff direction converges faster** — in fact its error is annihilated after a single iteration, because the step is tuned to land exactly at the minimiser along that axis. The flat direction retains a factor of 0.988999, so it needs $\ln(0.1)/\ln(0.988999) \approx 209$ iterations for each tenfold reduction in error. After the first step the error is essentially one-dimensional and the run time is set entirely by the small eigenvalue; this is why the convergence rate is governed by $\kappa(\mathbf{A}_\lambda)$, not by M alone.

At $\eta = 2.1/M$ the step overshoots: the factor in the stiff direction is $1 - 2.1 = -1.1$, of modulus $1.1 > 1$. With a nonzero initial component $c_{\text{stiff}}^{(0)}$ in that direction, $|c_{\text{stiff}}^{(k)}| = 1.1^k |c_{\text{stiff}}^{(0)}| \rightarrow \infty$, so $\|\mathbf{e}^{(k)}\| \rightarrow \infty$ and the method **diverges**, oscillating in sign across the valley with growing amplitude. That the flat direction is still contracting at 0.9769 is irrelevant: one expanding eigendirection is enough, and it grows by a factor of about 2.6 every ten steps. The step size is just past the boundary $2/M = 0.2200$ established above.

(c) Why conditioning is not a tuning rule

$\kappa(\mathbf{A}_\lambda) = (a_{\max} + \lambda)/(a_{\min} + \lambda)$ is strictly decreasing in λ and tends to 1 as $\lambda \rightarrow \infty$, so “choose λ to make gradient descent converge as fast as possible” has the degenerate answer $\lambda = \infty$. That limit sends $\beta_\lambda \rightarrow \mathbf{0}$: the fastest-converging problem is the one whose answer is the intercept-only model, carrying maximal bias and no predictive content at all. A criterion whose optimum is a model that ignores the data cannot be a selection criterion.

The deeper point is that the two quantities being confused live at different levels. Optimisation error $\|\beta^{(k)} - \beta_\lambda\|$ measures how well the algorithm has solved the problem, and κ controls only how fast it shrinks; statistical error measures how far β_λ itself is from the truth, or how badly the fitted model predicts new observations. Conditioning is a property of $\mathbf{A}_\lambda$ alone — it does not involve $\mathbf{y}$ at all — so it is blind to the bias-variance trade-off that λ actually governs. Choosing λ changes the estimand; choosing η and the iteration count changes only how accurately the estimand is computed. The correct division of labour is to fix λ by an out-of-sample criterion such as cross-validation or GCV, as in Question 5, and only then to use the resulting $\kappa(\mathbf{A}_\lambda)$ to pick a step size $\eta < 2/M$ or to decide that a direct solve of the $p \times p$ system is cheaper than iterating. Conditioning may inform the *grid* — very small λ can be numerically hard — but never the *choice* within it.

Question 5 · Selecting the ridge penalty for real-estate prediction

The two files are matched on `row_id`, giving $n_{\text{train}} = 332$ (ten folds of 33 or 34) and $n_{\text{test}} = 82$, with six quantitative predictors and `row_id` excluded. The grid is $\Lambda = \{0\} \cup \{10^{-4+0.05l}\}$ for $l = 0, \dots, 120$, i.e. 0 together with 121 values spanning 10^{-4} to 10^2 . Every fit standardises its predictors using only the rows available to it, and predicts on held-out rows using those same centres and scales; the test responses were untouched until all three penalties had been fixed.

```
for (i in 1:10) { hold <- folds == i                # standardise inside each fold split
  stf <- standardize_fit(Xtr[!hold, ]); Zf <- apply_std(Xtr[!hold, ], stf)
  Zv <- apply_std(Xtr[hold, ], stf)                # validation uses the training scales
  for (k in seq_along(lam_grid)) cv_err[i, k] <-
    mean((ytr[hold] - ridge_pred(Zv, ridge_fit(Zf, ytr[!hold], lam_grid[k])))^2) }
```

(a) Ten-fold cross-validation

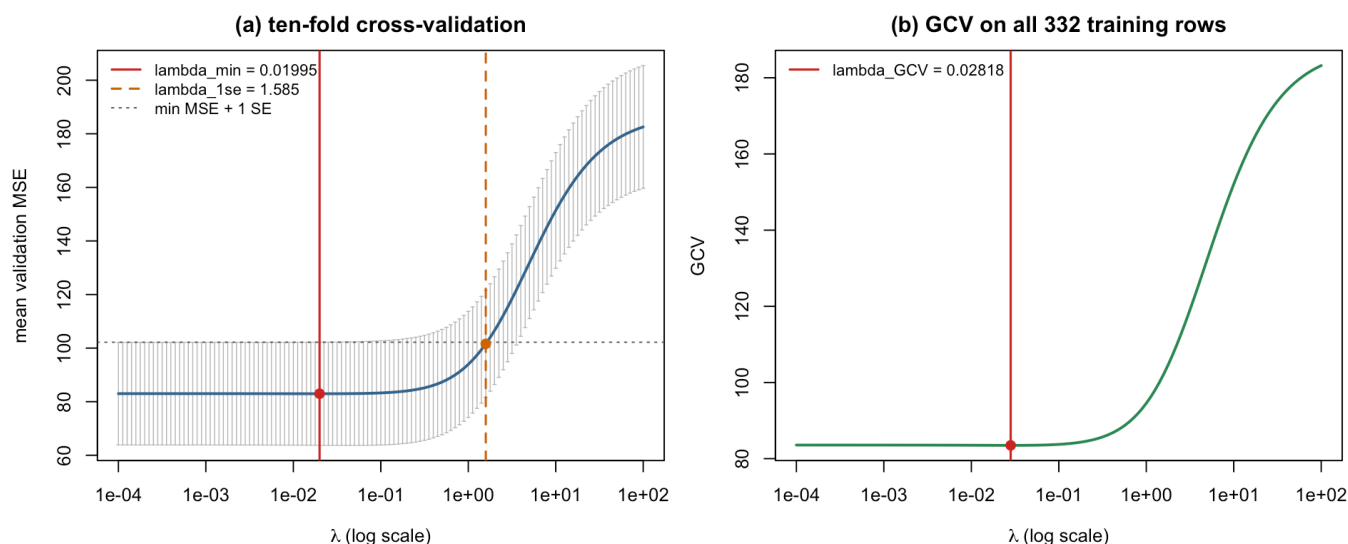


Figure 1. Left: mean validation MSE over the ten supplied folds, with ± 1 SE bars, $\lambda_{\min}$ (solid red), λ_{1se} (dashed orange) and the one-standard-error threshold (dotted). Right: the GCV curve on all 332 training rows. Both axes are logarithmic in λ ; the point $\lambda = 0$ cannot be drawn on a log scale and is reported in the text.

The minimum falls at $\lambda_{\min} = 10^{-1.7} = 0.019953$, with mean validation MSE 82.978 and $\text{SE}(\lambda_{\min}) = 19.230$. The one-standard-error threshold is therefore $82.978 + 19.230 = 102.208$, and the largest candidate in Λ meeting it is $\lambda_{1se} = 10^{0.2} = 1.5849$, whose mean validation MSE is 101.596.

Two features of the curve matter more than the location of the minimum. First, it is **extremely flat** at the left: across the whole range $\lambda \leq 0.1$ the mean validation MSE varies only between 82.978 and 83.275, a spread of 0.3 %, and OLS itself scores 83.012 — statistically indistinguishable from $\lambda_{\min}$. The exact grid point selected as $\lambda_{\min}$ is thus largely an artefact of which fold split was supplied. Second, the SE is enormous relative to the differences being resolved: 19.230 against a mean of 82.978, roughly 23 %, because a handful of unusually priced properties dominate whichever fold contains them.

The one-standard-error rule inherits that width, and here it reaches almost two orders of magnitude past $\lambda_{\min}$ before the curve finally rises through the threshold. The heuristic is meant to buy a simpler model at negligible cost in predicted error, but when the SE is this large it can license an aggressive amount of shrinkage — a point part (c) confirms.

(b) Generalised cross-validation

Means and scales are now estimated once from all 332 training rows and held fixed. For each λ , $\text{df}_{\text{eff}}(\lambda) = 1 + \sum_j d_j^2 / (d_j^2 + n\lambda)$, with d_j the singular values of the standardised training design (29.63, 18.85, 17.94, 14.23, 13.53, 7.17), and $\text{GCV}(\lambda) = \text{MSE}_{\text{train}}(\lambda) / \{1 - \text{df}_{\text{eff}}(\lambda)/n\}^2$. The curve is the right panel of Figure 1; its minimum is

$$\lambda_{\text{GCV}} = 10^{-1.55} = 0.028184, \quad \text{GCV} = 83.502, \quad \text{df}_{\text{eff}} = 6.689,$$

against $\text{GCV}(0) = 83.581$ for OLS. GCV lands within a factor of 1.4 of $\lambda_{\min}$ despite using a completely different mechanism — a closed-form leave-one-out approximation on all 332 rows rather than ten explicit refits — which is reassuring. Its curve is smooth and has no fold-to-fold sampling noise, but it is equally flat on the left for the same underlying reason.

(c) Refitting and the honest test evaluation

All three penalties were fixed before any test response was read. Each model is then refitted on all 332 training rows.

model	λ	df_{eff}	$\text{MSE}_{\text{train}}$	MSE_{test}	distance	latitude	longitude
OLS	0	7.000	80.094	66.322	-5.922	2.673	-0.157
ridge at $\lambda_{\min}$	0.019953	6.773	80.135	66.000	-5.572	2.704	0.108
ridge at $\lambda_{1\text{se}}$	1.584893	3.033	100.374	81.200	-2.257	1.731	1.464
ridge at λ_{GCV}	0.028184	6.689	80.171	65.903	-5.448	2.712	0.201

The last three columns are slopes on the standardised scale, shown for the three spatial predictors; date, age and stores behave similarly.

Training MSE rises monotonically with λ — 80.094, 80.135, 80.171, 100.374 — as it must, since OLS minimises it by construction. **Effective degrees of freedom** fall from the full 7 (six slopes plus intercept) to 6.77 and 6.69 at the two data-driven penalties, and collapse to 3.03 at $\lambda_{1\text{se}}$: the one-standard-error model has discarded more than half the flexibility of the design.

Test MSE tells the decisive story. $\lambda_{\min}$ and λ_{GCV} improve on OLS only marginally, $66.322 \rightarrow 66.000$ and 65.903 , gains of 0.5 % and 0.6 %, while $\lambda_{1\text{se}}$ is 22 % worse at 81.200. The explanation is that this design is barely collinear: its singular values range only from 29.63 to 7.17, a condition number of 4.13, against 29.7 in the Question 2 design. There is no near-null direction, so there is very little coefficient variance available for ridge to remove, and almost any shrinkage buys bias without a compensating variance reduction. Consistently with that, the coefficients barely move between OLS and λ_{GCV} ; the one exception is longitude, whose slope flips from -0.157 to +0.201, marking it as the weakly identified predictor (it is the one nearly redundant with latitude and distance as a description of location). At $\lambda_{1\text{se}}$ every slope is pulled roughly 60 % toward zero — distance from -5.92 to -2.26 — which is straightforward under-fitting, visible in both the training and test columns. Here the one-standard-error rule fails not because the rule is wrong but because $\text{SE}(\lambda_{\min}) = 19.23$ is far too wide a tolerance on a flat curve: the rule effectively asks how far λ can move before the curve rises by 23 %, and on this problem the answer is almost the entire grid.

Why the smallest test MSE in this table must not be used to select a model. The 82 test rows were set aside to provide one unbiased estimate of generalisation error for a model chosen without them. That guarantee is a statement about a *pre-specified* model: $E(\text{MSE}_{\text{test}})$ equals the true prediction error only when the model does not depend on those responses. The moment I scan the last column and keep the winner, the reported number becomes the minimum of four correlated random quantities, and a minimum is biased downward — the test set has been used for selection and has become a second validation set, with nothing independent left behind it. The bias is not hypothetical here, because the gaps are tiny

next to the noise. The standard error of the test MSE at λ_{GCV} , computed as the standard deviation of the 82 squared errors divided by $\sqrt{82}$, is **11.47**; even the paired standard error of the OLS – λ_{GCV} difference, which cancels the shared observations, is 0.454 against an observed gap of 0.420 — a ratio below 1. The apparent ordering $\lambda_{\text{GCV}} < \lambda_{\text{min}} < \text{OLS}$ is well inside sampling noise and would very likely reshuffle on a different 82 rows. Selecting on it would report an optimistic error for a ranking that is not real. The valid procedure is the one followed above: fix λ from training data alone, refit once, and read the corresponding test MSE as the honest estimate — here 65.903 for the GCV-selected model. If I wanted to compare the three selection *rules* rather than the three fitted models, the comparison would have to be made by nested resampling inside the training set, with the test set still held in reserve.

Appendix · explain-to-me skill

Complete contents of explain-to-me/SKILL.md, as installed:

```
---
name: explain-to-me
description: Use when I explicitly ask you to explain a STAT 432 homework question that I have
not yet solved. Do not use it when I ask for a solution, for code, or for a check of work I have
already done.
---

# Intent

I am a STAT 432 student. When I invoke this skill, I want to understand what a
homework question is asking before I attempt it. I do not want the answer, and a
premature answer costs me the exercise.

# Instructions

- Read the whole question, including any setup shared with earlier parts, before
  replying.
- State the goal of the question in two or three plain sentences, then list each
  lettered part as one line saying what I must produce: a derivation, a number, a
  table, a plot, or an interpretation.
- Keep the question's own notation and symbols. Define any symbol the question
  introduces without defining, such as an estimator, a norm, or a matrix
  dimension.
- Name the quantities I am asked to report and where they come from, but do not
  compute them.
- Point me to one section of the assigned Week 3 reading and say in one sentence
  why it applies: use
  [Ridge Regression: Stability Through Shrinkage]
  (https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression.html)
  for shrinkage, collinearity, scaling, and bias-variance, and
  [From a Penalized Objective to a Fitted Ridge Model]
  (https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimization-and-cross-validation.html)
  for optimization, cross-validation, GCV, and degrees of freedom. If you cannot
  open the page, say so rather than inventing a section.
- Flag anything in the question that is easy to misread, such as a fixed design,
  a reused noise draw, a scaling convention, or a quantity that must be chosen
  before the test responses are examined.
- Close with one suggested first step and one question asking what is still
  unclear.

# Restrictions

- Do not derive results, do not write or run solution code, and do not state
  final numerical answers.
- Do not restate the question verbatim as your explanation.
- Keep the reply under roughly 300 words unless I ask for more.
```